

LABORATORY OBSERVATION / RECORD

Course:	Full Stack Web Development	Course Code:	23CM4121
Experiment No.:	Observation Task 5	Date:	21-09-2026
Student Name:	M. Naga Doleswari Aruna Devi	Roll No.:	A24126552267

1. TITLE

Design and Develop an Interactive Product Calculator Webpage Using HTML, CSS and JavaScript

2. AIM

To design and develop a webpage titled “Interactive Product Calculator” that accepts a product name, quantity and price from the user, and using JavaScript functions and event listeners, dynamically calculates and displays the total price whenever the quantity or price changes — without refreshing the page — while validating the entered data.

3. OBJECTIVE

The objectives of this experiment are:

- To understand and use JavaScript event listeners (click, input, change) to detect user actions on a form.
- To read and process form input values using the Document Object Model (DOM).
- To perform arithmetic calculations dynamically in JavaScript based on live user input.
- To validate user input (empty product name, non-positive quantity, negative price) before performing a calculation.
- To update page content dynamically using innerHTML / textContent without reloading the page.
- To design a clean, responsive interface for the calculator using CSS.

4. THEORY

JavaScript allows a webpage to respond to user actions in real time without reloading the page. This is achieved using event listeners — functions registered on DOM elements with `addEventListener()` — that run whenever a specified event (such as `click`, `input` or `change`) occurs on that element.

The values entered by the user are read using `document.getElementById()` and the `.value` property. Numeric fields are converted from strings using `Number()`, and `isNaN()` is used to check whether a value is a valid number. Combining these checks with simple comparisons (e.g. `quantity > 0`, `price ≥ 0`) allows the form to be validated before any calculation is performed.

Once the input is validated, the total price is computed as `quantity × price`, and the result is written back into the page by setting the `innerHTML` of a result element, updating the visible content instantly and dynamically.

5. ALGORITHM / PROCEDURE

- 1 Create three files: `index.html`, `style.css` and `script.js`.

- 2 In index.html, build a container with a heading, a form containing Product Name, Quantity and Price inputs, a "Calculate total" button, and an empty result heading.
- 3 Link style.css for styling and script.js for the calculation logic.
- 4 In style.css, style the body, container, labels, inputs, button and result area.
- 5 In script.js, select the productName, quantity, price and result elements using getElementById.
- 6 Define a calculateTotal() function that reads the current values of the three inputs.
- 7 Inside calculateTotal(), validate the input: Product Name must not be empty; Quantity must be a number greater than 0; Price must be a number that is not negative.
- 8 If validation fails, display "Please enter valid details." in the result area.
- 9 If validation passes, compute total = quantity × price and display the Product, Quantity and Total Price.
- 10 Attach calculateTotal() to the button's click event, and to the input and change events of all three fields, so the total updates live as the user types.
- 11 Save the files and open index.html in a browser.
- 12 Test the page with valid and invalid combinations of inputs and record the observations against the expected results.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Interactive Product Calculator</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<header>
<h1>Interactive Product Calculator</h1>
</header>

<div class="container">
<h2>the Product calculator</h2>

<label for="productName">Product Name:</label>
<input type="text" id="productName" placeholder="Enter product name">

<label for="quantity">Quantity:</label>
<input type="number" id="quantity" placeholder="Enter quantity" min="1">

<label for="price">Price:</label>
<input type="number" id="price" placeholder="Enter price" min="0">

<button onclick="calculateTotal()">Calculate total</button>

<h2 id="result"></h2>
</div>

<script src="script.js"></script>
</body>
</html>
```

style.css

```
body {
font-family: Arial, sans-serif;
background-color: #f2f2f2;
```

```

}

.container {
width: 400px;
margin: 100px auto;
padding: 25px;
background-color: white;
border-radius: 10px;
box-shadow: 0 0 10px gray;
}

h1 {
text-align: center;
}

label {
display: block;
margin-top: 15px;
}

input {
width: 95%;
padding: 10px;
margin-top: 5px;
}

button {
margin-top: 20px;
padding: 10px 20px;
cursor: pointer;
}

#result {
margin-top: 20px;
}

```

script.js

```

const nameInput = document.getElementById('productName');
const qtyInput = document.getElementById('quantity');
const priceInput = document.getElementById('price');
const resultEl = document.getElementById('result');

function calculateTotal() {
const name = nameInput.value.trim();
const qty = Number(qtyInput.value);
const price = Number(priceInput.value);

const isValid =
name !== '' &&
!isNaN(qty) && qty > 0 &&
!isNaN(price) && price >= 0;

if (!isValid) {
resultEl.textContent = 'Please enter valid details.';
return;
}

const total = qty * price;
resultEl.innerHTML =
'Product: ' + name + '<br>' +
'Quantity: ' + qty + '<br>' +
'Total Price: \u20B9' + total;
}

// Recalculate live as the user types or changes a field
[nameInput, qtyInput, priceInput].forEach(el => {
el.addEventListener('input', calculateTotal);
el.addEventListener('change', calculateTotal);
});

```

```
});
```

7. CODE EXPLANATION

Tag / Statement	Description
<link rel="stylesheet" href="style.css">	Links the external CSS file used to style the calculator.
<input id="productName/quantity/price">	Form controls used to capture the product name, quantity and price entered by the user.
document.getElementById()	Selects the input and result elements so their values can be read or updated from script.js.
addEventListener('input'/'change', ...)	Registers calculateTotal() to run automatically whenever a field's value changes, enabling live recalculation without a page refresh.
Number(), isNaN()	Convert the text entered by the user into numbers and check whether the conversion produced a valid number.
Validation condition (name, qty > 0, price >= 0)	Ensures the product name is not empty and that quantity/price are sensible numbers before calculating the total.
resultEl.textContent = 'Please enter valid details.'	Displays a validation error message when any of the input checks fail.
total = qty * price	Computes the total price by multiplying the validated quantity and price.
resultEl.innerHTML = ...	Dynamically updates the result heading with the Product, Quantity and Total Price, without reloading the page.

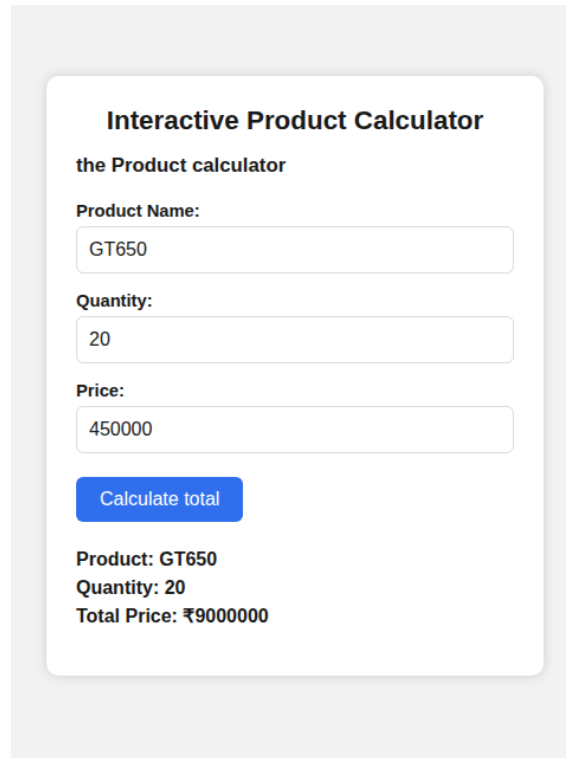
8. TEST CASES AND EXECUTION DATA

The page was opened in a browser and tested with the following product name, quantity and price combinations, each verified against the expected result:

Test Case	Product Name	Quantity	Price	Expected Result	Status
TC-01	GT650	20	450000	Total Price: ₹9000000	Pass
TC-02	Pen	5	10	Total Price: ₹50	Pass
TC-03	Laptop	2	50000	Total Price: ₹100000	Pass
TC-04	Book	3	250	Total Price: ₹750	Pass
TC-05	Mobile	1	20000	Total Price: ₹20000	Pass
TC-06	(empty)	2	100	Please enter valid details.	Pass
TC-07	Pen	0	10	Please enter valid details.	Pass
TC-08	Pen	5	-10	Please enter valid details.	Pass
TC-09	(empty)	0	0	Please enter valid details.	Pass

9. OUTPUT

Actual output produced by running the program above in a browser (shown here for the TC-01 input values):



The image shows a web form titled "Interactive Product Calculator" with the subtitle "the Product calculator". It contains three input fields: "Product Name:" with the value "GT650", "Quantity:" with the value "20", and "Price:" with the value "450000". Below these fields is a blue button labeled "Calculate total". At the bottom of the form, the results are displayed: "Product: GT650", "Quantity: 20", and "Total Price: ₹9000000".

Field	Value
Product Name	GT650
Quantity	20
Price	450000
Product	GT650
Quantity	20
Total Price	₹9000000

Fig. 9.1 — Interactive Product Calculator showing a valid calculation

10. RESULT

Thus, a webpage titled “Interactive Product Calculator” was successfully designed and developed using HTML, CSS and JavaScript. JavaScript functions and event listeners were used to dynamically calculate and display the total price whenever the quantity or price changed, without refreshing the page. Input validation for an empty product name, a non-positive quantity and a negative price was implemented and verified through browser testing, matching the expected results shown above.

LABORATORY OBSERVATION / RECORD

Course:	Full Stack Web Development	Course Code:	23CM4121
Experiment No.:	Observation Task 5 (Part A, Q2)	Date:	21-09-2026
Student Name:	M. Naga Doleswari Aruna Devi	Roll No.:	A24126552267

1. TITLE

Node.js File Management Using the File System (fs) Module

2. AIM

To develop a Node.js application that accepts a filename and content from the user and, using the File System (fs) module, creates/writes the file, reads its contents, appends additional content and displays the final contents of the file — while validating the entered data.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the Node.js runtime and how built-in modules are loaded using `require()`.
- To create and write a file using the File System module (`writeFileSync()`).
- To read the contents of a file (`readFileSync()`) and display them on the console.
- To append additional content to an existing file (`appendFileSync()`) without losing the old content.
- To accept the filename and content from the user at run time using the `readline` module.
- To validate user input (empty filename, empty content) and display suitable error messages.
- To observe the difference between overwriting a file (`write`) and appending to a file (`append`).

4. THEORY

Node.js is a JavaScript runtime that executes JavaScript outside the browser. Its built-in modules are loaded with `require()`. The File System module (fs) allows a program to work with files on the computer — to create, read, update and delete them.

The main methods used in this experiment are: `writeFileSync(file, data)` creates a new file, or overwrites the file if it already exists; `readFileSync(file, 'utf8')` returns the entire content of the file as a string; `appendFileSync(file, data)` adds data at the end of the file and keeps the existing content; and `existsSync(file)` checks whether a file is already present. The `Sync` versions wait until the operation is completed, which keeps the program simple; their asynchronous counterparts (`writeFile`, `readFile`, `appendFile`) use callbacks or promises.

The filename and content are accepted from the keyboard using the `readline` module, which reads lines from standard input (`process.stdin`). The input is validated with `trim()` so that an empty (or blank) filename or content is rejected before any file operation is performed, and the results are displayed on the console using `console.log()`.

5. ALGORITHM / PROCEDURE

- 1 Create a project folder and a file named app.js.
- 2 Import the fs and readline modules using require().
- 3 Create a readline interface and define a helper function ask() that displays a prompt and returns the line typed by the user.
- 4 Accept the filename from the user. If it is empty, display "Error: Filename is required" and stop.
- 5 Check whether the file already exists using fs.existsSync().
- 6 Accept the content from the user. If the content is empty and the file does not exist, display "Error: Content cannot be empty" and stop. If the content is empty but the file already exists, keep the existing file unchanged.
- 7 If content is entered, create/write the file using fs.writeFileSync() (a new file is created; an existing file is overwritten) and display a success message.
- 8 Read the file using fs.readFileSync() and display its contents.
- 9 Accept additional content from the user. If it is not empty, append it using fs.appendFileSync() and display a success message.
- 10 Read the file again with fs.readFileSync() and display the final contents.
- 11 Close the readline interface. Save the file and run it using the command: node app.js
- 12 Test the program with valid and invalid combinations of inputs and record the observations against the expected results.

6. PROGRAM

app.js

```
// File Management using the Node.js File System (fs) module
const fs = require('fs');
const readline = require('readline');

const rl = readline.createInterface({ input: process.stdin });
const lines = rl[Symbol.asyncIterator]();

// Show a prompt and wait for the user to type one line
async function ask(question) {
  process.stdout.write(question);
  const { value } = await lines.next();
  return value === undefined ? '' : value;
}

async function main() {
  console.log('==== Node.js File Management =====');

  // 1. Accept the filename and validate it
  const filename = (await ask('Enter filename: ')).trim();
  if (filename === '') {
    console.log('Error: Filename is required');
    return;
  }
  const fileExists = fs.existsSync(filename);

  // 2. Accept the content, then create / write the file
  const content = await ask('Enter content: ');
  if (content.trim() === '') {
    if (!fileExists) {
      console.log('Error: Content cannot be empty');
      return;
    }
    console.log('No new content entered. Existing file is kept as it is.');
```

```
  } else {
    fs.writeFileSync(filename, content);
```

```

    console.log(fileExists
      ? 'Existing file overwritten successfully.'
      : 'File created and content written successfully.');
```

```

  }

  // 3. Read the contents of the file
  console.log('\nContents of ' + filename + ':');
  console.log(fs.readFileSync(filename, 'utf8'));

  // 4. Accept additional content and append it
  const extra = await ask('\nEnter additional content to append (Enter to skip): ');
  if (extra.trim() !== '') {
    fs.appendFileSync(filename, extra);
    console.log('Additional content appended successfully.');
```

```

  }

  // 5. Display the final contents of the file
  console.log('\nFinal contents of ' + filename + ':');
  console.log(fs.readFileSync(filename, 'utf8'));
}

main()
  .catch(err => console.log('Error: ' + err.message))
  .finally(() => rl.close());

```

Run command: `node app.js`

7. CODE EXPLANATION

Tag / Statement	Description
<code>const fs = require('fs')</code>	Imports the built-in File System module used to create, read and append files.
<code>readline.createInterface({ input: process.stdin })</code>	Creates an interface that reads the text typed by the user from the keyboard (standard input).
<code>ask(question)</code>	Helper function that prints a prompt and returns the line entered by the user (used with <code>async/await</code>).
<code>filename.trim() === ''</code>	Validation: removes surrounding spaces and rejects an empty filename with “Filename is required”.
<code>fs.existsSync(filename)</code>	Checks whether the file already exists, so that the correct message is shown and an existing file can be overwritten or kept.
<code>content.trim() === ''</code>	Validation: rejects empty content with “Content cannot be empty” when the file does not already exist.
<code>fs.writeFileSync(filename, content)</code>	Creates the file and writes the content. If the file already exists, its old content is overwritten.
<code>fs.readFileSync(filename, 'utf8')</code>	Reads the whole file and returns its content as text, which is displayed using <code>console.log()</code> .
<code>fs.appendFileSync(filename, extra)</code>	Adds the additional content at the end of the file; the existing content remains unchanged.
<code>main().catch(...).finally(...)</code>	Runs the program, prints any file-system error message and always closes the readline interface.

8. TEST CASES AND EXECUTION DATA

The program was run from the command line (node app.js) and tested with the following filename and content combinations, each verified against the expected result:

Test Case	Input	Expected Output	Type	Status
TC-01	Filename: notes.txt Content: Hello Node.js	File is created and content is written successfully	✓ Valid	Pass
TC-02	Filename: student.txt Content: PBL Project	File is created with the given content	✓ Valid	Pass
TC-03	Filename: data.txt Content: Welcome Additional: " to Node.js"	Additional content is appended and final content is displayed	✓ Valid	Pass
TC-04	Filename: test.txt Content: (empty)	Display an error such as "Content cannot be empty"	✗ Invalid	Pass
TC-05	Filename: (empty) Content: Hello	Display "Filename is required"	✗ Invalid	Pass
TC-06	Filename: sample.txt Content: Hello	Existing file is overwritten if using writeFile / writeFileSync	✓ Valid	Pass
TC-07	Filename: sample.txt Additional: " World"	Existing content remains and " World" is appended	✓ Valid	Pass

Note: For TC-06 and TC-07, sample.txt already existed from an earlier run. In TC-07 the content prompt was left blank (Enter) so that the existing file is kept and only the additional content is appended. In TC-05 the program stops at the filename check, so the content prompt is not reached.

9. OUTPUT

Actual output produced by running the program above from the command line. TC-01 and TC-02 follow the same steps as TC-03 without the additional content, so TC-03 is shown as the complete flow:

```
●●● Terminal — Node.js

$ node app.js
===== Node.js File Management =====
Enter filename: data.txt
Enter content: Welcome
File created and content written successfully.

Contents of data.txt:
Welcome

Enter additional content to append (Enter to skip):  to Node.js
Additional content appended successfully.

Final contents of data.txt:
Welcome to Node.js
```

Fig. 9.1 — TC-03: file created, read, appended and final contents displayed (data.txt)

```
●●● Terminal — Node.js

$ node app.js
===== Node.js File Management =====
Enter filename: test.txt
Enter content:
Error: Content cannot be empty

●●● Terminal — Node.js

$ node app.js
===== Node.js File Management =====
Enter filename:
Error: Filename is required
```

Fig. 9.2 — TC-04 and TC-05: validation errors for empty content and empty filename

```
●●● Terminal — Node.js

$ node app.js
===== Node.js File Management =====
Enter filename: sample.txt
Enter content: Hello
Existing file overwritten successfully.

Contents of sample.txt:
Hello

Enter additional content to append (Enter to skip):

Final contents of sample.txt:
Hello

●●● Terminal — Node.js

$ node app.js
===== Node.js File Management =====
Enter filename: sample.txt
Enter content:
No new content entered. Existing file is kept as it is.

Contents of sample.txt:
Hello

Enter additional content to append (Enter to skip): World
Additional content appended successfully.

Final contents of sample.txt:
Hello World
```

Fig. 9.3 — TC-06 and TC-07: existing file overwritten, then appended (sample.txt)

10. RESULT

Thus, a Node.js application was successfully developed using the File System (fs) module. The program accepts a filename and content from the user and performs the file operations of creating/writing the file, reading its contents, appending additional content and displaying the final contents. Input validation for an empty filename and empty content was implemented and verified through command-line testing, matching the expected results shown above.

LABORATORY OBSERVATION / RECORD

Course:	Full Stack Web Development	Course Code:	23CM4121
Experiment No.:	Observation Task 5 (Part A, Q3)	Date:	21-09-2026
Student Name:	M. Naga Doleswari Aruna Devi	Roll No.:	A24126552267

1. TITLE

Dynamic Modification of Content, Style and Attributes of HTML Elements Using the Document Object Model (DOM) and JavaScript

2. AIM

To explain the Document Object Model (DOM) and to develop a webpage that demonstrates how JavaScript can dynamically change the content, style and attributes of an HTML element without reloading the page.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the DOM as a tree representation of an HTML document.
- To select HTML elements using `document.getElementById()`.
- To change element content using `innerHTML`.
- To change element style using the `style` property.
- To change element attributes using `setAttribute()`, `getAttribute()` and `removeAttribute()`.
- To trigger JavaScript functions from buttons using the `onclick` event.

4. THEORY

The **Document Object Model (DOM)** is a programming interface provided by the browser for HTML documents. When a page loads, the browser parses the HTML and builds a tree of objects called nodes: the document is the root, every tag becomes an element node, every attribute becomes an attribute node and every piece of text becomes a text node. JavaScript can read and modify this tree, and the browser instantly re-renders the page to match. Hence the page can be changed without reloading.

```
document
|-- html
    |-- head --- title
    |-- body
        |-- div.container
            |-- h1#title --- "Welcome to DOM"
            |-- p#message --- "This is the original content."
            |-- div.buttons --- button, button, button
```

JavaScript modifies an element in three ways. **Content** is changed by assigning a new value to `element.innerHTML`. **Style** is changed through `element.style.property` (CSS names are written in camelCase, e.g. `backgroundColor`), which sets inline CSS. **Attributes** such as `title`, `class`, `href` or `data-*` are changed with `setAttribute(name, value)`, read with `getAttribute(name)` and deleted with `removeAttribute(name)`. Elements are first located with `document.getElementById()`.

5. ALGORITHM / PROCEDURE

1. Create three files: `index.html`, `style.css` and `script.js`.

2. In index.html, create a container with a heading (id="title"), a paragraph (id="message") and three buttons: Change Content, Change Style and Change Attribute.
3. Link style.css and script.js to the HTML page.
4. In style.css, style the body, container, title, message and buttons (including a hover effect).
5. In script.js, define changeContent() which uses getElementById() and innerHTML to replace the heading and paragraph text.
6. Define changeStyle() which uses the style property to change colour, font size, font family, background colour and padding.
7. Define changeAttribute() which uses setAttribute() to set the title and data-status attributes of the message paragraph, and getAttribute() to confirm the new value.
8. Attach each function to its button using the onclick attribute.
9. Open index.html in a browser and click each button, observing the change without page reload.
10. Test with different click sequences and record the observations against the expected results.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>DOM Manipulation</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1 id="title">Welcome to DOM</h1>
    <p id="message">This is the original content.</p>
    <div class="buttons">
      <button onclick="changeContent()">Change Content</button>
      <button onclick="changeStyle()">Change Style</button>
      <button onclick="changeAttribute()">Change Attribute</button>
    </div>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

style.css

```
* { box-sizing: border-box; }
body {
  margin: 0; min-height: 100vh;
  display: flex; justify-content: center; align-items: center;
  font-family: Arial, sans-serif;
  background: linear-gradient(135deg, #dbeafe, #f3e8ff);
}
.container {
  width: 600px; padding: 40px; text-align: center;
  background-color: white; border-radius: 20px;
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.15);
}
#title { color: #2563eb; font-size: 40px; margin-bottom: 25px; }
#message { padding: 15px; font-size: 18px; background-color: #f1f5f9;
  border-radius: 8px; margin-bottom: 25px; }
.buttons { display: flex; justify-content: center; gap: 15px; }
button {
  padding: 12px 22px; border: none; border-radius: 8px;
  background-color: #2563eb; color: white;
  font-size: 15px; cursor: pointer;
}
button:hover { background-color: #1d4ed8; }
```

script.js

```
// Function to change the content
function changeContent() {
  document.getElementById("title").innerHTML =
    "Content Changed Successfully!";
  document.getElementById("message").innerHTML =
    "The content has been changed using JavaScript.";
}

// Function to change the style
function changeStyle() {
  document.getElementById("title").style.color = "#7c3aed";
  document.getElementById("title").style.fontSize = "45px";
  document.getElementById("title").style.fontFamily = "Georgia, serif";
  document.getElementById("message").style.backgroundColor = "#fce7f3";
  document.getElementById("message").style.color = "#831843";
  document.getElementById("message").style.fontSize = "20px";
  document.getElementById("message").style.fontFamily = "Georgia, serif";
  document.getElementById("message").style.padding = "20px";
}

// Function to change the attributes
function changeAttribute() {
  var msg = document.getElementById("message");
  msg.setAttribute("title", "Attribute changed using JavaScript");
  msg.setAttribute("data-status", "updated");
  console.log(msg.getAttribute("title")); // read the new value
}
```

7. CODE EXPLANATION

Tag / Statement	Description
document.getElementById("title")	Selects the element with id "title" from the DOM so it can be modified.
.innerHTML = "..."	Replaces the content inside the selected element; the page updates instantly.
.style.color / .fontSize / .backgroundColor	Sets inline CSS properties of the element (CSS property names are written in camelCase).
.setAttribute("title", "...")	Adds a new attribute or changes its value if it already exists.
.getAttribute("title")	Reads the current value of an attribute.

Tag / Statement	Description
onclick="changeContent()"	Event attribute that calls the JavaScript function when the button is clicked.
display:flex; justify-content:center	CSS used to centre the container on the page and place the buttons in a row.
button:hover	CSS rule that darkens the button colour when the mouse is over it.

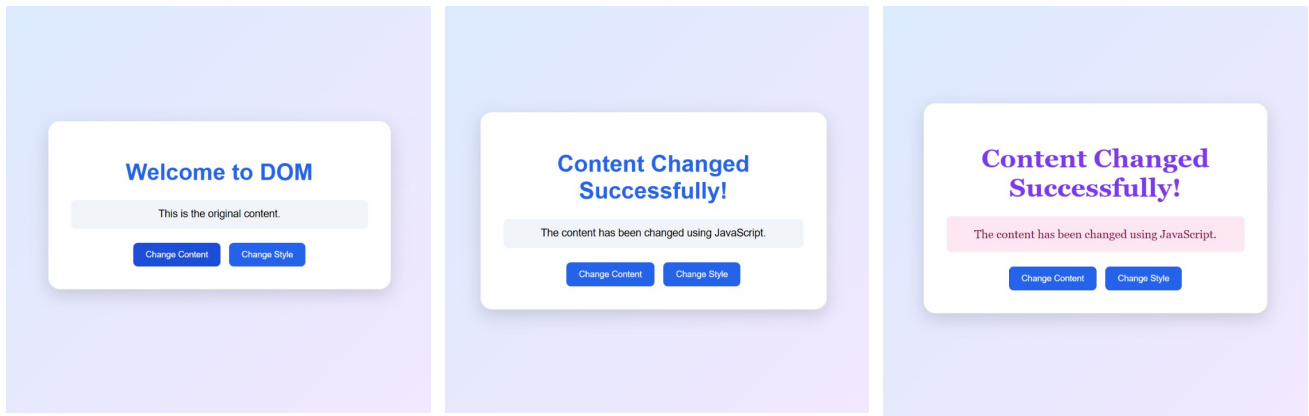
8. TEST CASES AND EXECUTION DATA

The page was opened in a browser and the buttons were clicked in different sequences. Each observation was verified against the expected result:

Test Case	Action	Expected Result	Status
TC-01	Open the page (no click)	Title "Welcome to DOM" (blue, 40px); message "This is the original content." on light grey background	Pass
TC-02	Click Change Content	Title becomes "Content Changed Successfully!"; message becomes "The content has been changed using JavaScript."; styles unchanged	Pass
TC-03	Click Change Style	Title turns purple (#7c3aed), 45px, Georgia; message gets pink background (#fce7f3), dark red text, 20px, padding 20px	Pass
TC-04	Click Change Attribute	Message gets title = "Attribute changed using JavaScript" (tooltip on hover) and data-status = "updated"; visible text unchanged	Pass
TC-05	Click Change Content, then Change Style	Both the new text and the new style are shown together	Pass
TC-06	Click Change Style, then Change Content	Text changes but the purple/Georgia style is retained (innerHTML changes only content)	Pass
TC-07	Click Change Content twice	Same result as one click; content is replaced, not duplicated	Pass
TC-08	Click Change Attribute twice	Attribute values are overwritten, not duplicated	Pass
TC-09	Hover the mouse over a button	Button colour changes from #2563eb to #1d4ed8	Pass
TC-10	Refresh the page after any change	All changes are lost; the page returns to the original state (DOM changes are not permanent)	Pass

9. OUTPUT

Actual output produced by running the program in a browser: (a) original page, (b) after clicking Change Content, (c) after clicking Change Style.



(a) Original page

(b) Change Content

(c) Change Style

Fig. 9.1 — DOM manipulation: original page, changed content and changed style

10. RESULT

Thus, the Document Object Model (DOM) was studied and a webpage was successfully developed using HTML, CSS and JavaScript. JavaScript functions were used to dynamically change the content (innerHTML), style (style property) and attributes (setAttribute) of HTML elements without reloading the page, and the behaviour was verified through browser testing, matching the expected results shown above.

LABORATORY OBSERVATION / RECORD

Course:	Full Stack Web Development	Course Code:	23CM4121
Experiment No.:	Observation Task 5 (Part A, Q4)	Date:	21-09-2026
Student Name:	M. Naga Doleswari Aruna Devi	Roll No.:	A24126552267

1. TITLE

Express.js Student Server: Creating a Basic Web Server with Routes (/, /students, /about) Using Node.js and Express.js

2. AIM

To develop a basic Express.js server with the routes /, **/students** and **/about**, where the /students route returns a list of at least five students in JSON format and the /about route displays information about the application, using appropriate HTTP methods and routing.

3. OBJECTIVE

The objectives of this experiment are:

- To understand Express.js as a web framework built on Node.js.
- To create a server using `express()` and start it with `app.listen()`.
- To define routes using the HTTP GET method with `app.get()`.
- To send HTML responses using `res.send()` and JSON responses using `res.json()`.
- To store student data as an array of objects and return it from a route.
- To handle invalid URLs using a fallback middleware that returns the status code 404.

4. THEORY

Express.js is a minimal and flexible web application framework for Node.js. It simplifies creating servers and handling HTTP requests. The framework is installed with `npm install express` and loaded using `require("express")`. Calling `express()` returns an application object (usually named `app`) used to define routes and start the server.

Routing means deciding how the application responds to a client request for a particular URL (path) and HTTP method. A route has the form `app.METHOD(path, handler)`, for example `app.get("/students", (req, res) => {...})`. The handler receives the request object `req` and the response object `res`. The GET method is used here because the routes only retrieve data (POST would create, PUT update and DELETE remove data).

The response is sent using `res.send()` (text or HTML) or `res.json()` (converts a JavaScript object/array to JSON and sets the header `Content-Type: application/json`). Routes are checked in the order in which they are written, so a middleware written with `app.use()` after all the routes runs only when no route matches; it is used here to return the status 404 (Not Found). Finally, `app.listen(PORT, callback)` starts the server on the given port (3000).

5. ALGORITHM / PROCEDURE

1. Create a project folder, open a terminal in it and run `npm init -y` to create `package.json`.
2. Install Express using `npm install express`.
3. Create a file named `app.js` and import `express`; create the `app` object and set `PORT = 3000`.
4. Create an array named `students` containing five student objects (`id`, `name`, `branch`).
5. Define the GET route `"/` that sends a heading and a welcome message.
6. Define the GET route `"/students` that returns the `students` array using `res.json()`.

7. Define the GET route `"/about"` that sends information about the application.
8. Define a fallback middleware using `app.use()` that sends status 404 with the message `"404 - Page Not Found"` for any other URL.
9. Start the server using `app.listen()` and display the server URL on the console.
10. Run the program using `node app.js` and open `http://localhost:3000` in a browser.
11. Visit each route, test invalid URLs and record the observations against the expected results.

6. PROGRAM

app.js

```
const express = require("express");
const app = express();
const PORT = 3000;

// Student data
const students = [
  { id: 1, name: "Arun", branch: "CSE" },
  { id: 2, name: "Priya", branch: "AIML" },
  { id: 3, name: "Rahul", branch: "ECE" },
  { id: 4, name: "Sita", branch: "CSE" },
  { id: 5, name: "Kiran", branch: "AIML" }
];

// Home route
app.get("/", (req, res) => {
  res.send(`
    <h1>Student Management Server</h1>
    <p>Welcome to the Express.js Student Application</p>
  `);
});

// Students route
app.get("/students", (req, res) => {
  res.json(students);
});

// About route
app.get("/about", (req, res) => {
  res.send(`
    <h1>About Application</h1>
    <p>This application is developed using Node.js and Express.js.</p>
    <p>It provides student information through different routes.</p>
  `);
});

// Invalid route
app.use((req, res) => {
  res.status(404).send("404 - Page Not Found");
});

// Start server
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

Run command: `node app.js`

7. CODE EXPLANATION

Tag / Statement	Description
<code>const express = require("express")</code>	Imports the Express.js framework installed through npm.
<code>const app = express()</code>	Creates the Express application object used to define routes and start the server.

Tag / Statement	Description
<code>const students = [...]</code>	Array of five objects; each has id, name and branch (the student data returned by /students).
<code>app.get("/", (req, res) => {...})</code>	Route for the home page; handles GET requests to "/" and sends an HTML heading and welcome message using <code>res.send()</code> .
<code>app.get("/students", ...)</code>	Route that returns the students array as JSON using <code>res.json()</code> .
<code>app.get("/about", ...)</code>	Route that sends an HTML page describing the application.
<code>app.use((req, res) => {...})</code>	Fallback middleware placed after all routes; runs only for unmatched URLs and returns <code>res.status(404).send(...)</code> .
<code>app.listen(PORT, callback)</code>	Starts the server on port 3000 and prints the URL on the console when it is ready.

8. TEST CASES AND EXECUTION DATA

The server was started using `node app.js` and the routes were requested from the browser (and with other HTTP methods where indicated). Each observation was verified against the expected result:

Test Case	Request	Expected Result	Status
TC-01	GET /	Heading "Student Management Server" and text "Welcome to the Express.js Student Application"	Pass
TC-02	GET /students	JSON array containing five students: Arun (CSE), Priya (AIML), Rahul (ECE), Sita (CSE), Kiran (AIML)	Pass
TC-03	GET /students (check response)	Content-Type is application/json; every object has id, name and branch; total 5 records	Pass
TC-04	GET /about	Page "About Application" describing that it is developed using Node.js and Express.js and provides student information	Pass
TC-05	GET /contact (undefined route)	Status 404 with message "404 - Page Not Found"	Pass
TC-06	POST /students	Status 404 (only the GET method is defined for this route)	Pass
TC-07	GET /students/	Trailing slash is ignored by default; the same student list is returned	Pass
TC-08	GET /Students	Route matching is case-insensitive by default; the same student list is returned	Pass
TC-09	Run <code>node app.js</code>	Console displays "Server running at http://localhost:3000"	Pass

9. OUTPUT

Actual output produced by running the program: the console message, the home page (route /) and the JSON list returned by the /students route.

```
$ node app.js
Server running at http://localhost:3000
```

Fig. 9.1 — Server started on port 3000

Student Management Server

Welcome to the Express.js Student Application

Fig. 9.2 — Route "/" (home page)

```
Pretty-print ☐  
[{"id":1,"name":"Arun","branch":"CSE"}, {"id":2,"name":"Priya","branch":"AIML"}, {"id":3,"name":"Rahul","branch":"ECE"}, {"id":4,"name":"Sita","branch":"CSE"}, {"id":5,"name":"Kiran","branch":"AIML"}]
```

Fig. 9.3 — Route "/students" returning five student records in JSON format

10. RESULT

Thus, a basic Express.js server was successfully developed using Node.js. The routes "/", "/students" and "/about" were created using the GET method: the /students route returns a list of five students in JSON format, the /about route displays information about the application, and any undefined URL returns a 404 error message. The behaviour was verified through browser testing, matching the expected results shown above.